ИТОГОВОЕ ЗАДАНИЕ (Дифференцированный зачет)

Дисциплина: Технология доступа к базам данных ADO.NET
Формат сдачи: Практическая работа + защита кода (ответы на вопросы).

Предметная область: «Интернет-магазин электроники»

Схема базы данных:

- **Categories**: `Id (int, PK)`, `Name (nvarchar)`
- **Products**: `Id (int, PK)`, `Name (nvarchar)`, `CategoryId (int, FK)`, `Price (decimal)`, `Stock (int)`
- **Customers**: `Id (int, PK)`, `FullName (nvarchar)`, `Email (nvarchar)`, `RegistrationDate (datetime)`
- **Orders**: `Id (int, PK)`, `CustomerId (int, FK)`, `OrderDate (datetime)`, `TotalAmount (decimal)`
- **OrderItems**: `Id (int, PK)`, `OrderId (int, FK)`, `ProductId (int, FK)`, `Quantity (int)`

Подготовительный этап (Общий для всех заданий)

1. Создайте базу данных и таблицы согласно схеме. Заполните таблицы тестовыми данными (минимум по 5-10 записей в каждой).
2. Создайте новый проект (Console App или Windows Forms/WPF) на языке C#.
3. **Требование безопасности:** Все запросы к БД должны быть параметризованными (защита от SQL-инъекций). Весь код работы с БД обязательно оборачивается в блоки `try-catch-finally` с выводом понятных сообщений об ошибках.

ЗАДАНИЕ 1. Присоединенный режим, Конфигурации и Асинхронность

Цель: Продемонстрировать навыки работы с базовыми классами ADO.NET, выноса настроек и асинхронного программирования.

Пошаговые требования:

1. **Конфигурация:** Создайте файл конфигурации (`App.config` или `appsettings.json`). Вынесите туда строку подключения (Connection String). Напишите код для чтения этой строки при запуске приложения.
2. **Фабрика провайдеров:** Создайте класс-хелпер для работы с БД. Вместо прямого создания `SqlConnection`, используйте `DbProviderFactories.GetFactory("System.Data.SqlClient")` (или другого провайдера) для получения объектов `DbConnection` и `DbCommand`.
3. **Метод 1 (Добавление):** Напишите синхронный метод добавления нового товара. Он должен принимать название, ID категории, цену и количество, и выполнять `INSERT` через `ExecuteNonQuery()`.
4. **Метод 2 (JOIN-запрос):** Напишите метод, который выводит в консоль (или ListBox) список всех товаров в формате: `[Категория] Название товара - Цена (Остаток: X шт.)`. Используйте SQL-запрос с `JOIN` и чтение через `DbDataReader`.
5. **Асинхронный метод:** Напишите асинхронный метод `Task<int> GetStockAsync(int productId)`, который возвращает остаток конкретного товара. Используйте `ExecuteScalarAsync()`.

ЗАДАНИЕ 2. Отсоединенный режим

Цель: Продемонстрировать работу с `DataSet`, `DataTable` и механизмом синхронизации изменений.

Пошаговые требования:

1. Напишите метод, который с помощью `DbDataAdapter` загружает таблицу `Products` в `DataTable`.
2. Реализуйте логику изменения цен: программно пройдитесь по всем строкам `DataTable`, где `CategoryId` равен заданному, и увеличьте `Price` на 10%.
3. Программно добавьте новую строку (`DataRow`) в `DataTable` (новый товар).
4. **Синхронизация:** Настройте `InsertCommand` и `UpdateCommand` у адаптера вручную (без использования `CommandBuilder`). Вызовите метод `Update()` для сохранения изменений в БД.
5. *Дополнительно (на оценку "отлично"):* Реализуйте удаление строки из `DataTable` и настройте `DeleteCommand`.

ЗАДАНИЕ 3. LINQ to SQL

Цель: Научиться сопоставлять объектную модель с реляционной БД с помощью атрибутов и выполнять запросы.

Пошаговые требования:

1. Создайте классы-сущности для таблиц `Products`, `OrderItems`, `Categories` (или других, необходимых для запроса).
2. Разметьте классы и свойства атрибутами: `[Table(Name = "Products")]`, `[Column(IsPrimaryKey = true)]`, `[Association(...)]`.
3. Создайте класс-наследник `DataContext`, передав в конструктор строку подключения.
4. **Запрос:** Напишите LINQ-запрос (query syntax или method syntax), который находит **Топ-3 самых продаваемых товаров за последние 30 дней**.
   - *Подсказка:* Вам нужно соединить `OrderItems`, `Orders` и `Products`, отфильтровать по дате (`OrderDate`), сгруппировать по названию товара, просуммировать `Quantity` и отсортировать по убыванию.
5. Выведите результат в консоль: `Название товара — Продано штук`.

ЗАДАНИЕ 4. Entity Framework (Code-First)

Цель: Освоить современный ORM, работу со связями и управление транзакциями.

Пошаговые требования:

1. Подключите NuGet-пакет Entity Framework (или EF Core).
2. Опишите классы сущностей (`Product`, `Order`, `OrderItem` и т.д.) и настройте навигационные свойства (например, `public virtual ICollection<OrderItem> OrderItems { get; set; }`).
3. Создайте класс `ShopContext : DbContext` с `DbSet<>` для каждой таблицы. Настройте строку подключения.
4. Выполните инициализацию БД (через миграции Code-First или метод `Database.EnsureCreated()`).
5. **Бизнес-логика (Оформление заказа):** Напишите метод `PlaceOrder(int customerId, Dictionary<int, int> productsToBuy)`.
   - Метод должен создавать новый `Order` и связанные с ним `OrderItem`.
   - **Критически важно:** Перед созданием заказа проверьте, хватает ли товара на складе (`Stock`). Если товара не хватает — выбросьте исключение (`Exception`).
   - Если товара хватает — уменьшите `Stock` и сохраните изменения.
   - **Обязательное требование:** Оберните все операции с БД в явную транзакцию (`using var transaction = context.Database.BeginTransaction();`). В случае любой ошибки (например, отрицательный остаток) должен происходить `Rollback()`.

ЗАДАНИЕ 5. Использование Dapper (Микро-ORM)

Цель: Оценить легкость и производительность "легковесных" ORM для сложных аналитических запросов.

Пошаговые требования:

1. Подключите NuGet-пакет `Dapper`.
2. Создайте DTO-класс (Data Transfer Object) для результата запроса, например: `class SleepingCustomerDto { public string FullName {get; set;} public string Email {get; set;} public DateTime RegistrationDate {get; set;} }`.
3. **Аналитический запрос:** Напишите метод, который с помощью Dapper выполняет SQL-запрос и возвращает список DTO.
   - **Условие запроса:** Вывести список «спящих» клиентов (FullName, Email, RegistrationDate). Это клиенты, которые зарегистрировались более 3 месяцев назад, но **не сделали ни одного заказа** (используйте `LEFT JOIN` с таблицей `Orders` и условие `WHERE Orders.Id IS NULL`).
4. Используйте метод расширения `connection.Query<SleepingCustomerDto>(sqlQuery)`.

Требования к сдаче и отчету:

1. Исходный код проекта (архив или ссылка на Git-репозиторий).
2. SQL-скрипт для создания базы данных и заполнения ее тестовыми данными.
3. Скриншоты работающего приложения (консольный вывод или интерфейс), подтверждающие выполнение каждого из 5 заданий.

Вопросы к зачету

Блок 1. Введение в ADO.NET и архитектура

1. Что такое ADO.NET и какую основную задачу он решает?
2. Из каких основных компонентов состоит архитектура ADO.NET?
3. В чем принципиальная разница между присоединенной (Connected) и отсоединенной (Disconnected) моделями доступа к данным?
4. Что такое строка подключения (Connection String)? Какие основные параметры она содержит?
5. Что такое пул подключений (Connection Pooling) и как он влияет на производительность приложения?

Блок 2. Присоединенный режим и Фабрика провайдеров

6. Для чего используется объект `DbConnection` (или `SqlConnection`)? Почему его важно оборачивать в блок `using`?
7. Что такое `DbProviderFactory` и какую проблему решает использование этого паттерна?
8. В чем преимущество использования абстрактных классов (`DbConnection`, `DbCommand`) перед конкретными реализациями (`SqlConnection`, `SqlCommand`)?
9. Какие основные методы объекта `DbCommand` вы знаете и для чего предназначен каждый из них (`ExecuteNonQuery`, `ExecuteScalar`, `ExecuteReader`)?
10. Что такое `DataReader`? Почему этот объект обеспечивает максимальную скорость чтения данных?
11. В чем отличие потокового чтения данных через `DataReader` от загрузки в `DataTable` с точки зрения потребления оперативной памяти?
12. Какие типы данных может возвращать метод `ExecuteScalar`? Приведите примеры задач, где его уместно использовать.
13. Что такое параметризованные запросы и как они защищают приложение от SQL-инъекций (SQL Injection)?
14. Как реализовать асинхронное выполнение запросов к базе данных в ADO.NET? Какие методы для этого используются?
15. Как вынести строку подключения в конфигурационный файл (например, `appsettings.json` или `App.config`) и прочитать ее в коде?

Блок 3. Отсоединенный режим (Disconnected Mode)

16. Для чего предназначены объекты `DataSet` и `DataTable`?
17. Что такое `DataAdapter`? Какие ключевые методы он использует для заполнения таблиц и сохранения изменений (`Fill`, `Update`)?
18. Как в `DataTable` отслеживаются изменения строк? Что такое `DataRowState` (какие состояния вы знаете)?
19. Что такое `CommandBuilder` и в каких случаях его целесообразно использовать?
20. Какие ограничения существуют при использовании `CommandBuilder` для автоматической генерации SQL-команд?
21. Опишите пошагово процесс изменения данных в `DataTable` и их последующей синхронизации с базой данных.

Блок 4. LINQ to SQL

22. Что такое LINQ (Language Integrated Query) и какие преимущества он дает разработчику перед написанием сырых SQL-запросов?
23. Что такое `DataContext` в LINQ to SQL и за какую функциональность он отвечает?
24. Какие атрибуты используются для маппинга (сопоставления) классов C# на таблицы базы данных (например, `[Table]`, `[Column]`)?
25. Как в LINQ to SQL настраиваются связи между сущностями (один-ко-многим)? Какой атрибут для этого используется?

26. Что такое «отложенное выполнение» (Deferred Execution) в LINQ и как оно работает?

Блок 5. Entity Framework (EF)

27. Что такое Entity Framework? Какие основные подходы к разработке существуют (Code-First, Database-First, Model-First)?
28. Что такое `DbContext` и `DbSet`? Как они связаны с таблицами базы данных?
29. Как в Entity Framework работает механизм отслеживания изменений (Change Tracker) и метод `SaveChanges()`?
30. Что такое миграции (Migrations) в Entity Framework и зачем они нужны?
31. В чем разница между ленивой загрузкой (Lazy Loading), жадной загрузкой (Eager Loading через `Include`) и явной загрузкой (Explicit Loading)?
32. Как в Entity Framework реализовать явную транзакцию (например, при оформлении сложного заказа) и зачем нужен `Rollback()`?

Блок 6. Dapper (Микро-ORM)

33. Что такое Dapper? К какому классу библиотек он относится?
34. Какие основные методы предоставляет Dapper для выполнения запросов (`Query`, `QueryFirstOrDefault`, `Execute`)?
35. Как Dapper маппит (сопоставляет) результаты SQL-запроса в объекты C#?
36. В каких сценариях разработки лучше выбрать Dapper, а не Entity Framework? Приведите примеры.

Блок 7. Сравнение, безопасность и общие практики

37. В чем разница между `List<T>` и `DataTable`? В каких ситуациях предпочтительнее использовать каждый из них?
38. Что такое транзакция в базах данных? Какими свойствами (ACID) она должна обладать?
39. Сравните производительность классического ADO.NET, Dapper и Entity Framework. От чего зависит скорость их работы?
40. Какие основные классы исключений используются при работе с базами данных в .NET (например, `DbException`)? Как правильно организовывать обработку ошибок (`try-catch-finally`) при работе с БД?